

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Ячменников Егор

Группа
J3214

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать моковое API средствами JSON-сервера и подключить к нему авторизацию.

Ход работы

В ходе работы было написано 6 js-скриптов и тематические страницы (концерты, фестивали, театр).

1. filter.js

Этот скрипт отвечает за фильтрацию на событий на страницах index.html, concerts.html, festivals.html, theatre.html.

```
document.addEventListener('DOMContentLoaded', () => {
  const API_BASE = 'http://localhost:3000';
  const container = document.getElementById('events-container');
  const filterForm = document.getElementById('filter-form');

  if (!container) return;

  const pageDefaultType = container.getAttribute('data-type');

  async function fetchAndRenderEvents() {
    try {
      const serverParams = {};
      if (pageDefaultType) {
        serverParams.type = pageDefaultType;
      } else {
        const typeSelect = document.getElementById('filter-type');
        if (typeSelect && typeSelect.value && typeSelect.value !== 'Все типы') {
          serverParams.type = typeSelect.value;
        }
      }

      const response = await axios.get(`${API_BASE}/events`, { params: serverParams });
      let events = response.data;

      const dateInput = document.getElementById('filter-date');
      if (dateInput && dateInput.value) {
        events = events.filter(event => event.dateRaw === dateInput.value);
      }

      const placeInput = document.getElementById('filter-place');
      if (placeInput && placeInput.value.trim() !== '') {
        const searchString = placeInput.value.trim().toLowerCase();
        events = events.filter(event => {
          return event.place && event.place.toLowerCase().includes(searchString);
        });
      }
    }
  }
});
```

Сначала вешаем прослушивание события при полной загрузке страницы. Затем задаём базовый URL сервера и получаем элементы с формой фильтрации и контейнер, в котором отображаются события.

Далее получаем текущий (начальный) тип событий (он задаётся атрибутом 'data-type', на главной странице его нет, на странице концертов он равен "concert" и т.д.). И пишем функцию, которая будет получать и вставлять нужные события.

Если у страницы есть фильтр на тип события по умолчанию (тематическая страница) будем передавать его в запрос на сервер, иначе получаем фильтр из блока фильтрации и передаём его. Полученные события записываем в переменную *events*.

Далее получаем фильтр на дату события. Здесь всё просто, ищем по точному совпадению даты.

Затем на место события. Место будем искать как подстрочку поля *place* в базе данных событий.

```
container.innerHTML = '';

if (events.length === 0) {
  container.innerHTML = `
    <div class="col-12 text-center text-muted py-5">
      <h5>По вашему запросу ничего не найдено</h5>
    </div>
  `;
  return;
}

events.forEach(event => {
  const cardHtml = `
    <div class="col">
      <div class="card h-100 event-card shadow-sm">
        
        <div class="card-body">
          <h5 class="card-title">${event.title}</h5>
          <p class="text-muted mb-1">${event.date}</p>
          <p class="card-text text-secondary">${event.faceDescription}</p>
          <div class="d-flex justify-content-between align-items-center">
            <span class="fw-bold fs-5">от ${event.price} Р</span>
            <a href="event.html?id=${event.id}" class="btn btn-outline-primary">Купить билет</a>
          </div>
        </div>
      </div>
    </div>
  `;
  container.insertAdjacentHTML('beforeend', cardHtml);
});

} catch (error) {
  console.error('Ошибка при обработке мероприятий:', error);
  container.innerHTML = `<div class="col-12 alert alert-danger text-center">Ошибка загрузки данных.</div>`;
}
```

Затем переходим к вставке. На всякий случай предварительно очищаем текст контейнера. Если ни одно событие не подходит под заданные фильтры, отображаем соответствующее сообщение. Иначе проходим циклом по всем найденным событиям и вставляем их в контейнер, подставляя в шаблон данные из БД. В случае ошибки на каком-либо из этапов отображаем соответствующее сообщение.

```

fetchAndRenderEvents();

const resetBtn = document.getElementById('btn-reset');

if (resetBtn && filterForm) {
  resetBtn.addEventListener('click', () => {
    filterForm.reset();

    fetchAndRenderEvents();
  });
}

if (filterForm) {
  filterForm.addEventListener('submit', (e) => {
    e.preventDefault();
    fetchAndRenderEvents();
  });
}
});

```

Вызываем функцию при загрузке страницы (подтянет события по дефолтному типу).

Также добавляем обработчик нажатия кнопки сброса фильтров и их применения (preventDefault отменяет перезагрузку страницы).

2. events.js

Этот скрипт отвечает за сборку страницы для каждого мероприятия (event.html).

```

document.addEventListener('DOMContentLoaded', async () => {
  const API_BASE = 'http://localhost:3000';

  const urlParams = new URLSearchParams(window.location.search);
  const eventId = urlParams.get('id');

  if (!eventId) {
    alert('Мероприятие не выбрано!');
    window.location.href = 'index.html';
    return;
  }

  let selectedSeatName = "Ряд 1, Место 1";

  const seats = document.querySelectorAll('.seat:not(.occupied)');

  if (seats.length > 0) {
    seats[0].classList.add('selected');
    if (seats[0].hasAttribute('title')) {
      selectedSeatName = seats[0].getAttribute('title');
    }
    document.getElementById('modal-selected-seat').innerText = selectedSeatName;
  }

  seats.forEach(seat => {
    seat.addEventListener('click', (e) => {
      seats.forEach(s => s.classList.remove('selected'));
      e.target.classList.add('selected');

      selectedSeatName = e.target.getAttribute('title') || "Выбранное место";

      document.getElementById('modal-selected-seat').innerText = selectedSeatName;
    });
  });
});

```

Сначала получаем id мероприятия из параметров URL (передавали в прошлом скрипте - `<a href="event.html?id=${event.id}" class="btn btn-outline-primary">Купить билет</a>`). Затем делаем фокус на первом незанятом месте в зале (для удобства, пользователь может выбрать любое сам). Делаем обработчик нажатия на место: удаляем класс `selected` с предыдущего выбранного места, добавляем его текущему выбранному месту, сохраняем номер места (если его вдруг нет, ставим заглушку 'Выбранное место') и записываем это в модальное окно покупки.

```

try {
  const response = await axios.get(`${API_BASE}/events/${eventId}`);
  const event = response.data;

  document.getElementById('page-title').innerText = event.title + " — Билеты.py";
  document.getElementById('type-page-link').innerText = event.typePage;
  document.getElementById('type-page-link').href = event.typePageLink;
  document.getElementById('current-page').innerText = event.title;
  document.getElementById('event-title').innerText = event.title;
  document.getElementById('event-description').innerText = event.fullDescription;
  document.getElementById('event-image').src = event.image;

  document.getElementById('sidebar-date').innerText = event.date;
  document.getElementById('sidebar-place').innerText = event.place;
  document.getElementById('sidebar-price').innerText = `от ${event.price.toLocaleString()} Р`;

  document.getElementById('modal-event-title').innerText = event.title;
  document.getElementById('modal-event-place').innerText = event.place;
  document.getElementById('modal-total-price').innerText = `${event.price.toLocaleString()} Р`;
} catch (error) {
  console.error('Ошибка при получении данных:', error);
  alert('Не удалось загрузить данные мероприятия. Проверьте json-server.');
```

```

  document.getElementById('order-form').addEventListener('submit', async (e) => {
    e.preventDefault();
  });
});

```

Заполняем страницу данными из БД. И вешаем событие на кнопку покупки билета (заглушка).

3. auth.js

Скрипт, отвечающий за аутентификацию.

```

if (registerForm) {
  registerForm.addEventListener('submit', async (e) => {
    e.preventDefault();
    registerError.classList.add('data-none', 'd-none');

    const name = document.getElementById('reg-name').value.trim();
    const email = document.getElementById('reg-email').value.trim().toLowerCase();
    const password = document.getElementById('reg-password').value;
    const role = document.getElementById('reg-role').value;

    try {
      const checkUserResponse = await axios.get(`${API_BASE}/users`, {
        params: { email: email }
      });

      if (checkUserResponse.data.length > 0) {
        showError(registerError, 'Пользователь с таким Email уже зарегистрирован');
        return;
      }

      const newUser = { name, email, password, role };
      const createResponse = await axios.post(`${API_BASE}/users`, newUser);

      if (createResponse.status === 201) {
        localStorage.setItem('currentUser', JSON.stringify(createResponse.data));

        if (createResponse.data.role === 'organizer') {
          window.location.href = 'org_profile.html';
        } else {
          window.location.href = 'index.html';
        }
      }
    } catch (error) {
      console.error('Ошибка при регистрации:', error);
      showError(registerError, 'Произошла ошибка на сервере. Попробуйте позже.');
```

Логика регистрации. Прячем блоки для отображения ошибок. Получаем введённые пользователем данные. Проверяем по email нет ли уже такого пользователя. Иначе записываем его в БД. Если запись произошла успешно, записываем в localStorage данные авторизованного пользователя. Редиректим пользователя в зависимости от его роли.

```

if (loginForm) {
  loginForm.addEventListener('submit', async (e) => {
    e.preventDefault();
    loginError.classList.add('d-none');

    const email = document.getElementById('login-email').value.trim().toLowerCase();
    const password = document.getElementById('login-password').value;

    try {
      const response = await axios.get(`${API_BASE}/users`, {
        params: { email: email }
      });

      const users = response.data;

      if (users.length === 0) {
        showError(loginError, 'Неверный email или пароль');
        return;
      }

      const user = users[0];

      if (String(user.password) !== String(password)) {
        showError(loginError, 'Неверный email или пароль');
        return;
      }

      localStorage.setItem('currentUser', JSON.stringify(user));

      if (user.role === 'organizer') {
        window.location.href = 'org_profile.html';
      } else {
        window.location.href = 'index.html';
      }
    } catch (error) {
      console.error('Ошибка при авторизации:', error);
      showError(loginError, 'Произошла ошибка на сервере. Попробуйте позже.');
```

Логика входа. Делаем запрос в БД по email. Берём первого попавшегося пользователя (если нашли хоть кого-то). Далее проверяем пароль (отправлять запрос на сервер сразу с паролем не стал, т.к. если пароль состоит только из чисел, сервер преобразует его к числу, и скажет что у пользователя пароль другой, даже если он такой же). Если всё совпало записываем в localStorage. Делаем редирект.

4. check_auth.js

Скрипт для проверки авторизации, нужен для отрисовки профиля авторизованного пользователя в шапке и функционала кнопки 'Выйти'.

```
const currentUser = JSON.parse(localStorage.getItem('currentUser'));

if (currentUser.role === "organizer") {
  const nameBlock = document.getElementById('org-name');

  nameBlock.innerHTML = `Организатор: ${currentUser.name}`
} else {
  const authNavContainer = document.getElementById('auth-btn');
  if (currentUser && authNavContainer) {
    authNavContainer.innerHTML = `
      <span class="navbar-text me-3">Привет, <a href="user_profile.html"><strong>${currentUser.name}</strong></a></span>
      <button class="btn btn-outline-danger btn-sm" id="btn-logout">Выйти</button>
    `;
  }
}

document.getElementById('btn-logout').addEventListener('click', () => {
  localStorage.removeItem('currentUser');
  window.location.href = 'index.html';
});
```

Получаем данные пользователя из localStorage. Если это организатор, подставляем его имя в шапку. Иначе (пользователь) делаем то же самое, только для него (плюс ссылка на страницу профиля) и кнопку 'Выйти' (у организации она сразу есть на странице, т.к. она не подразумевается для неавторизованного пользователя).

Делаем функционал для кнопки выхода - удаление данных из localStorage и редиректим на главную.

5. user_profile_script.js

Скрипт для страницы профиля пользователя. На данный момент отрисовывает предстоящие и прошедшие события.

```

const upcomingContainer = document.getElementById('upcoming');
const pastContainer = document.getElementById('past');

document.addEventListener('DOMContentLoaded', async () => {
  const currentUser = JSON.parse(localStorage.getItem('currentUser'));
  const API_BASE = 'http://localhost:3000';

  try {
    const [ordersResponse, eventsResponse] = await Promise.all([
      axios.get(`${API_BASE}/orders`, { params: { "userId": currentUser.id } }),
      axios.get(`${API_BASE}/events`)
    ]);

    const rawOrders = ordersResponse.data;
    const events = eventsResponse.data;

    const orders = rawOrders.map(order => {
      const foundEvent = events.find(e => e.id === order.eventId);
      return {
        ...order,
        event: foundEvent || null
      };
    });

    document.getElementById('tickets-amount').innerText = `Билетов: ${orders.length}`;

    upcomingContainer.innerHTML = "";
    pastContainer.innerHTML = "";

    const now = new Date();

    const upcomingOrders = orders.filter(order => {
      if (!order.event || !order.event.dateRaw) return false;
      return new Date(order.event.dateRaw) >= now;
    });

    const pastOrders = orders.filter(order => {
      if (!order.event || !order.event.dateRaw) return false;
      return new Date(order.event.dateRaw) < now;
    });
  }
});

```

Получаем контейнеры для предстоящих и прошедших событий. Затем запрашиваем заказы пользователя по его id и все события. Связываем события с заказами: `map` меняет применяет к каждому элементу `rawOrders` заданную функцию и возвращает новый массив; ищем для каждого заказа соответствующее событие, возвращаем заказ (`...order` - все поля `order`) + событие.

Распределяем заказы на предстоящие и прошедшие.

```

if (upcomingOrders.length === 0) {
  upcomingContainer.innerHTML = `
    <div class="col-12 text-center text-muted py-5">
      <h5>По вашему запросу ничего не найдено</h5>
    </div>
  `;
}

if (pastOrders.length === 0) {
  pastContainer.innerHTML = `
    <div class="col-12 text-center text-muted py-5">
      <h5>По вашему запросу ничего не найдено</h5>
    </div>
  `;
}

upcomingOrders.forEach(order => {
  const event = order.event;
  const eventDate = new Date(event.dateRaw);
  const cardHtml = `
    <div class="card ticket-card shadow-sm mb-3">
      <div class="card-body">
        <div class="row align-items-center">
          <div class="col-md-2 text-center border-end">
            <div class="h3 mb-0 fw-bold text-primary">${eventDate.getDate()}</div>
            <div class="small text-uppercase text-muted">${eventDate.toLocaleString('ru-RU', { month: 'short' })}</div>
          </div>
          <div class="col-md-6 ps-4">
            <h5 class="mb-1">${event.title}</h5>
            <p class="text-muted mb-0 small">${event.place}</p>
          </div>
          <div class="col-md-4 text-md-end mt-3 mt-md-0">
            <button class="btn btn-sm btn-light border">Заказ <strong>${order.id}</strong></button>
            <button class="btn btn-sm btn-outline-danger">Вернуть</button>
          </div>
        </div>
      </div>
    </div>
  `;
  upcomingContainer.insertAdjacentHTML('beforeend', cardHtml);
});

```

Добавляем карточки заказов в соответствующие контейнеры (для прошедших аналогично).

Вывод

В ходе лабораторной работы я научился писать js-скрипты и реализовывать моковое API (имитирующее реальное API) средствами JSON-сервера.